SENTINELSHIELD

Advanced Intrusion Detection & Web Protection System

PROJECT REPORT:

Prepared By:
Pugazhenthi A

Project:
SentinelShield – Advanced Intrusion Detection & Web Protection System

GitHub Repository:
https://github.com/pugazh-10/SentinelShield-WAF

Deployment URL:
https://sentinelshield-waf-bb8q.onrender.com

Date:
02-06-2026

ABSTRACT

SentinelShield is a lightweight Web Application Firewall (WAF) and Intrusion Detection System (IDS) developed using Python and Flask. The system inspects incoming HTTP requests, detects malicious payloads through signature-based analysis, blocks suspicious traffic, performs rate limiting, generates structured attack logs, and visualizes threats through a Security Operations Center (SOC) style dashboard.

The project was designed to demonstrate practical cybersecurity concepts including request inspection, attack detection, intrusion monitoring, threat logging, abuse prevention, and dashboard-based analytics.

---

1. INTRODUCTION

Web applications are among the most common targets of cyberattacks. Attackers frequently attempt to exploit vulnerabilities such as SQL Injection, Cross-Site Scripting (XSS), Command Injection, and Directory Traversal attacks to gain unauthorized access to systems.

Modern organizations deploy Web Application Firewalls (WAFs) and Intrusion Detection Systems (IDS) to identify and mitigate these threats.

SentinelShield was developed as a lightweight security solution that demonstrates the core functionality of these systems in a practical and educational environment.

---

2. PROBLEM STATEMENT

Many small web applications lack dedicated security controls capable of:

• Inspecting incoming requests

• Detecting malicious payloads

• Monitoring suspicious behavior

• Limiting abusive traffic

• Logging security incidents

• Visualizing attack activity

This project addresses these challenges by implementing a lightweight security monitoring and protection framework.

---

3. OBJECTIVES

Primary Objectives

• Inspect incoming HTTP requests

• Detect common web attacks

• Block malicious requests

• Monitor abusive traffic behavior

• Generate structured security logs

• Provide dashboard-based attack analytics

Secondary Objectives

• Understand WAF architecture

• Learn IDS workflows

• Gain practical cybersecurity implementation experience

---

4. PROJECT OVERVIEW

SentinelShield consists of five major modules:

1. Request Inspection Engine

2. Attack Detection Engine
3. Rate Limiting Module
4. Security Logging Module
5. SOC Dashboard

The system processes incoming requests and determines whether they should be allowed or blocked based on predefined attack signatures and behavioral rules.

---

5. TECHNOLOGY STACK

Technology	Purpose
Python	Core Development
Flask	Web Framework
Regex	Signature Matching
HTML/CSS	Dashboard Interface
Chart.js	Attack Analytics
JSON	Security Logging
GitHub	Version Control
Render	Cloud Deployment

---

6. SYSTEM ARCHITECTURE

Request
↓
Request Inspection
↓
Detection Engine
↓
Allow / Block Decision
↓
Security Logger
↓
SOC Dashboard

Description:

Incoming requests are analyzed before processing. Malicious requests are blocked and recorded. Security events are then visualized through the monitoring dashboard.

---

7. MODULE DESCRIPTION

7.1 Request Inspection Module

Purpose:

Capture and analyze incoming HTTP requests.

Collected Information:

• Request Parameters

• Source IP Address

• Request Method

• Payload Content

Output:

Parsed request information forwarded to the detection engine.

7.2 Attack Detection Engine

Purpose:

Identify malicious request patterns using signature-based detection.

Supported Attacks:

• SQL Injection

• Cross-Site Scripting (XSS)

• Command Injection

• Directory Traversal

• Local File Inclusion

Detection Technique:

Regular Expression (Regex) Pattern Matching

7.3 Rate Limiting Module

Purpose:

Prevent brute-force and flooding attacks.

Functionality:

Tracks request frequency from individual IP addresses and blocks excessive traffic beyond configured thresholds.

7.4 Security Logging Module

Purpose:

Maintain records of security events.

Logged Information:

- Timestamp
- Source IP
- Attack Type
- Severity Level
- Payload Information

Example:

Attack Type: SQL Injection

Severity: HIGH

IP: 127.0.0.1

7.5 SOC Dashboard

Purpose:

Provide real-time visibility into attack activity.

Dashboard Features:

- Total Threats Detected
- Attack Analytics
- Top Attacker IPs

• Recent Security Events

• Firewall Status

---

8. ATTACK DETECTION METHODOLOGY

SQL Injection

Example Payload:

' OR 1=1 --

Purpose:

Prevent database manipulation attacks.

Severity:

HIGH

Cross-Site Scripting (XSS)

Example Payload:

Purpose:

Prevent malicious script execution.

Severity:

MEDIUM

Command Injection

Example Payload:

; cat /etc/passwd

Purpose:

Prevent unauthorized system command execution.

Severity:

CRITICAL

Directory Traversal

Example Payload:

../../etc/passwd

Purpose:

Prevent unauthorized file access.

Severity:

HIGH

Local File Inclusion

Example Payload:

php://input

Purpose:

Prevent local file exploitation.

Severity:

HIGH

Rate Limiting Abuse

Purpose:

Prevent flooding and brute-force behavior.

Severity:

MEDIUM

---

9. IMPLEMENTATION

Project Structure

SentinelShield/

├— detector/

├— limiter/

├— logger/

├— logs/

├── templates/

├── screenshots/

├── app.py

├── requirements.txt

└── README.md

GitHub Repository:

https://github.com/pugazh-10/SentinelShield-WAF

Deployment:

https://sentinelshield-waf-bb8q.onrender.com

---

10. TESTING AND VALIDATION

Test Case 1

Attack:

SQL Injection

Input:

' OR 1=1 --

Expected Result:

Blocked

Actual Result:

Blocked

Status:

PASS

Test Case 2

Attack:

XSS

Input:

Expected Result:

Blocked

Actual Result:

Blocked

Status:

PASS

Test Case 3

Attack:

Directory Traversal

Input:

../../etc/passwd

Expected Result:

Blocked

Actual Result:

Blocked

Status:

PASS

Test Case 4

Attack:

Rate Limiting Abuse

Expected Result:

Blocked

Actual Result:

Blocked

Status:

PASS

---

11. RESULTS AND ANALYSIS

Based on attack simulations and generated security logs.

Total Logged Attacks:

19

Detected Attack Categories:

• SQL Injection

• XSS

• Directory Traversal

• Rate Limit Exceeded

Detection Accuracy:

95%

False Positives:

1

False Negatives:

0

Observations:

The system successfully detected the majority of simulated attacks and generated structured security records for further analysis.

---

12. DASHBOARD ANALYTICS

The SOC dashboard provides:

• Threat Monitoring

• Firewall Status

• Attack Distribution

• Recent Security Events

• Top Attacker IP Statistics

The dashboard enables administrators to quickly identify suspicious activity and analyze attack trends.

---

13. DEPLOYMENT

Repository:

https://github.com/pugazh-10/SentinelShield-WAF

Cloud Platform:

Render

Live Application:

https://sentinelshield-waf-bb8q.onrender.com

Deployment Benefits:

- Remote accessibility
- Demonstration capability
- Real-world hosting experience

---

14. CHALLENGES FACED

- Designing effective regex signatures
- Managing attack logging
- Implementing dashboard analytics
- Handling deployment configuration
- Testing multiple attack scenarios
- Managing cloud-hosted application behavior

---

15. KEY LEARNINGS

- Flask Web Development
- Web Application Security
- Intrusion Detection Concepts
- Request Inspection Techniques
- Security Logging
- Dashboard Development
- GitHub Workflow

• Cloud Deployment using Render

---

16. FUTURE ENHANCEMENTS

• Machine Learning-Based Detection

• Real-Time Alerts

• Database Storage

• Threat Intelligence Integration

• SIEM Integration

• User Authentication

• API Security Monitoring

• Advanced Behavioral Analysis

---

17. CONCLUSION

SentinelShield successfully demonstrates the core functionality of a lightweight Web Application Firewall and Intrusion Detection System. The project performs request inspection, attack detection, rate limiting, structured security logging, and dashboard-based monitoring.

Through practical implementation and testing, the project achieved high detection accuracy and provided hands-on experience with modern cybersecurity defense mechanisms.

The project serves as a foundation for future enhancements involving machine learning, real-time monitoring, and advanced threat analysis.

REFERENCES

1. OWASP Top 10
2. Flask Documentation
3. Python Documentation
4. OWASP Web Security Testing Guide
5. Render Deployment Documentation
6. GitHub Documentation